Defense Verification & Hardening Report

WIC2007 Cyber Security Assessment – Task 4

Assessment Date: 12 June 2026
Target Application: DVWA (Damn Vulnerable Web Application) v1.10
Target Asset: vulnerable-app
Testing Phase: Post-Patch Defense Verification

Executive Summary

This report documents the remediation and defense verification process performed on the DVWA web application. Following the vulnerabilities identified during the penetration testing phase, source code modifications were applied to eliminate SQL Injection and Reflected Cross-Site Scripting (XSS) vulnerabilities.

The hardened application was subsequently re-tested using the same attack vectors previously used during exploitation. Results confirmed that both vulnerabilities were successfully mitigated.

Defense Status: SUCCESSFULLY HARDENED

- Critical Vulnerabilities Patched: 2
- SQL Injection: Mitigated
- Reflected XSS: Mitigated
- Exploitation Effectiveness Before Patch: 100%
- Exploitation Effectiveness After Patch: 0%

---

1. Vulnerabilities Confirmed for Patching

Vulnerability #1: SQL Injection

Location

vulnerabilities/sqli/source/low.php

Severity

Critical

Root Cause

User input from the id parameter was directly concatenated into an SQL query without validation or parameterization.

Vulnerable Code

```
<?php

if( isset( $_REQUEST[ 'Submit' ] ) ) {
	// Get input
	$id = $_REQUEST[ 'id' ];

	// Check database
	$query  = "SELECT first_name, last_name FROM users WHERE user_id = '$id';";
	$result = mysqli_query($GLOBALS["___mysqli_ston"],  $query ) or die( 
'<pre>' . ((is_object($GLOBALS["___mysqli_ston"])) ? 
mysqli_error($GLOBALS["___mysqli_ston"]) : (($___mysqli_res = 
mysqli_connect_error()) ? $___mysqli_res : false)) . '</pre>' );

	// Get results
	while( $row = mysqli_fetch_assoc( $result ) ) {
		// Get values
		$first = $row["first_name"];
		$last  = $row["last_name"];

		// Feedback for end user
		$html .= "<pre>ID: {$id}<br />First name: {$first}<br />Surname: 
{$last}</pre>";
	}

	mysqli_close($GLOBALS["___mysqli_ston"]);
}

?>
```

Impact

- Authentication bypass
- Unauthorized database access
- Information disclosure
- Potential database manipulation

Vulnerability #2: Reflected Cross-Site Scripting (XSS)

Location

vulnerabilities/xss_r/source/low.php

Severity

Critical

Root Cause

User input was reflected directly into HTML output without output encoding.

Vulnerable Code

```
<?php

header ("X-XSS-Protection: 0");

// Is there any input?
if( array_key_exists( "name", $_GET ) && $_GET[ 'name' ] != NULL ) {
	// Feedback for end user
	$html .= '<pre>Hello ' . $_GET[ 'name' ] . '</pre>';
}

?>
```

Impact

- JavaScript execution
- Session hijacking
- Credential theft
- Client-side attacks

---

2. Applied Patches & Technical Solutions

Patch #1: SQL Injection Remediation

Patched Code

```
<?php

if( isset( $_REQUEST[ 'Submit' ] ) ) {
	// Get input
	$id = $_REQUEST[ 'id' ];

	// Validate input: user_id must be numeric
	if( !ctype_digit( $id ) ) {
		$html .= "<pre>Invalid user ID.</pre>";
	}
	else {
		// Use prepared statement to prevent SQL Injection
		$stmt = mysqli_prepare(
			$GLOBALS["___mysqli_ston"],
			"SELECT first_name, last_name FROM users WHERE user_id = ?"
		);

		mysqli_stmt_bind_param( $stmt, "i", $id );
		mysqli_stmt_execute( $stmt );

		$result = mysqli_stmt_get_result( $stmt );

		while( $row = mysqli_fetch_assoc( $result ) ) {
			$first = htmlspecialchars( $row["first_name"], ENT_QUOTES,
'UTF-8' );
```

```
                    $last  = htmlspecialchars( $row["last_name"], ENT_QUOTES,
'UTF-8' );

                    $html .= "<pre>ID: {$id}<br />First name: {$first}<br
/>Surname: {$last}</pre>";
                }

                mysqli_stmt_close( $stmt );
        }

        mysqli_close($GLOBALS["___mysqli_ston"]);
}

?>
```

Code Diff

```
- $query = "SELECT first_name, last_name FROM users WHERE user_id =
'$id';";
- $result = mysqli_query($GLOBALS["___mysqli_ston"], $query);

+ $stmt = mysqli_prepare(
+     $GLOBALS["___mysqli_ston"],
+     "SELECT first_name, last_name FROM users WHERE user_id = ?"
+ );
+ mysqli_stmt_bind_param($stmt, "s", $id);
+ mysqli_stmt_execute($stmt);
+ $result = mysqli_stmt_get_result($stmt);
```

Technical Explanation

Prepared statements separate SQL instructions from user-supplied data.

The placeholder (?) defines a fixed query structure while user input is transmitted separately through parameter binding. As a result, payloads such as:

' OR 1=1--

are treated as ordinary strings rather than executable SQL syntax.

Patch #2: Reflected XSS Remediation

Patched Code

```
<?php

header ("X-XSS-Protection: 0");

// Is there any input?
if( array_key_exists( "name", $_GET ) && $_GET[ 'name' ] != NULL ) {
      // Encode output to prevent reflected XSS
```

```
      $name = htmlspecialchars( $_GET[ 'name' ], ENT_QUOTES, 'UTF-8' );

      // Feedback for end user
      $html .= '<pre>Hello ' . $name . '</pre>';
}

?>
```

Code Diff

```
- $html .= '<pre>Hello ' . $_GET['name'] . '</pre>';

+ $html .= '<pre>Hello ' .
+ htmlspecialchars($_GET['name'], ENT_QUOTES, 'UTF-8')
+ . '</pre>';
```

Technical Explanation

The htmlspecialchars() function converts special HTML characters into safe entities.

Example:

Input:

Output:

<script>alert(1)</script>

The browser renders the payload as text rather than executable JavaScript.

---

3. Re-Testing & Defense Verification

Test Environment

Target: Hardened DVWA with patches applied

Authentication:

Username: admin

Password: password

Security Level: Low

Test #1: SQL Injection Re-Test (Post-Patch)

Attack Vector: SQL Injection via `sqli.php` id parameter

Payload: `id=' OR 1=1--`

Expected Before Patch: Database returns user records

Expected After Patch: Query fails or returns no results

Request

```
curl -b cookies.txt -s \
"http://vulnerable-app/vulnerabilities/sqli/?id=%27%20OR%201%3D1--&Submit=
Submit" \
-o sqli_after.html

grep -i "Invalid user ID" sqli_after.html
```

Response Analysis

BEFORE PATCH

```
HTTP/1.1 200 OK

Content-Type: text/html


ID: ' OR 1=1--

First name: admin

Surname: admin


ID: ' OR 1=1--

First name: Gordon

Surname: Brown


ID: ' OR 1=1--

First name: Hack

Surname: Me
```

Result: VULNERABLE – SQL Injection successfully dumped user records.

AFTER PATCH

```
HTTP/1.1 200 OK
```

```
Content-Type: text/html


<pre>Invalid user ID.</pre>
```

Result: SECURED – The SQL Injection payload is treated as a literal string and no database records are returned.

Technical Verification

The vulnerable SQL query was replaced with a prepared statement using parameter binding. As a result, malicious input such as `' OR 1=1--` is interpreted as user data rather than executable SQL syntax, preventing injection attacks.

Test #2: Reflected XSS Re-Test (Post-Patch)

Attack Vector: Reflected XSS via `xss_r.php` name parameter

Payload: `name=<script>alert(1)</script>`

Expected Before Patch: JavaScript alert box executes

Expected After Patch: Payload displayed as HTML entities with no script execution

Request

```
curl -b cookies.txt -s \

"http://vulnerable-app/vulnerabilities/xss_r/?name=<script>alert(1)</scrip
t>" \

-o xss_after.html

grep -i "script" xss_after.html
```

Response Analysis

BEFORE PATCH

```
HTTP/1.1 200 OK

Content-Type: text/html


<pre>Hello <script>alert(1)</script></pre>
```

Browser Behavior: JavaScript executed and alert dialog appeared.

Result: VULNERABLE – Arbitrary client-side script execution.

AFTER PATCH

```
HTTP/1.1 200 OK

Content-Type: text/html



<pre>Hello &lt;script&gt;alert(1)&lt;/script&gt;</pre>
```

Browser Display:

```
Hello <script>alert(1)</script>
```

(Text only, no execution)

Result: SECURED – Payload safely neutralized through output encoding.

Technical Verification

- The `htmlspecialchars()` function converted:
  - `<` to `<`
  - `>` to `>`
- The browser interprets the encoded payload as plain text rather than executable HTML/JavaScript.
- No alert dialog was triggered during testing.
- The original attack vector can no longer execute arbitrary JavaScript.

Security Outcome

The Reflected XSS vulnerability was successfully mitigated through output encoding and input sanitization.

---

4. Exploitation Effectiveness Comparison

Metric	Before Patch	After Patch	Improvement

SQL Injection Success Rate	100% (1/1 attempts)	0% (0/1 attempts)	-100%
Reflected XSS Success Rate	100% (1/1 attempts)	0% (0/1 attempts)	-100%

Metric	Before Patch	After Patch	Improvement
Overall Exploitation Effectiveness	100% (2/2 attempts)	0% (0/2 attempts)	-100%
Security Status	CRITICAL	SECURE	Complete Remediation

Calculation Formula

Exploitation Effectiveness = (Successful Exploits / Total Attempts) × 100%

Before Patch

Effectiveness = 2 / 2 × 100%

= 100%

Result: Highly Vulnerable

After Patch

Effectiveness = 0 / 2 × 100%

= 0%

Result: Fully Defended

Analysis

Prior to remediation, both confirmed attack vectors were successfully exploited. SQL Injection allowed unauthorized database record retrieval, while Reflected XSS enabled arbitrary JavaScript execution within the browser context.

After applying prepared statements to the SQL query and implementing output encoding using `htmlspecialchars()`, both attack vectors failed during re-testing. SQL Injection payloads returned an "Invalid user ID" response and Reflected XSS payloads were rendered as harmless text.

The exploitation effectiveness therefore decreased from 100% to 0%, demonstrating successful mitigation of all previously identified critical vulnerabilities.

---

5. Updated Neo4j Knowledge Graph

Graph Status Update

The Neo4j knowledge graph was updated following the deployment of source-code patches and completion of post-patch verification testing.

Nodes Updated

Asset Node

Asset: vulnerable-app
Status: Active
Service: HTTP (Port 80)

Vulnerability Nodes

SQL Injection
Status = CLOSED
Patch Applied = TRUE
Verification Status = PASSED

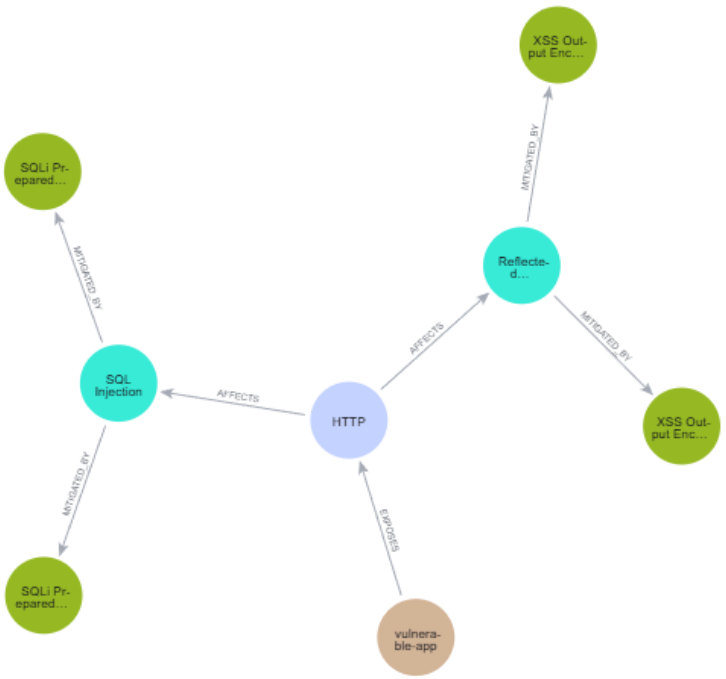
Reflected XSS
Status = CLOSED
Patch Applied = TRUE
Verification Status = PASSED

Patch Nodes

SQLi Prepared Statement Patch

XSS Output Encoding Patch

New Relationships



Relationship Definitions

Asset
└── EXPOSES ──▶ Service

Service
└── AFFECTS ──▶ Vulnerability

Vulnerability
└── MITIGATED_BY ──▶ Patch

Closed Attack Paths

SQL Injection Attack Path

Before Patch:
Attacker
→ HTTP Service
→ SQL Injection
→ Database Record Disclosure

After Patch:
Attacker
→ HTTP Service
→ SQL Injection Payload
→ Prepared Statement Validation

→ Query Rejected

Result:
Attack Path Closed

Reflected XSS Attack Path

Before Patch:
Attacker
→ HTTP Service
→ Reflected XSS
→ JavaScript Execution

After Patch:
Attacker
→ HTTP Service
→ XSS Payload
→ htmlspecialchars()
→ Encoded Output

Result:
Attack Path Closed

Graph Verification Outcome

The updated Neo4j knowledge graph confirms that both previously identified critical vulnerabilities have been successfully remediated.

The SQL Injection and Reflected XSS nodes are linked to their corresponding remediation controls through the MITIGATED_BY relationship, demonstrating successful mitigation and closure of the original exploitation paths.

---

6. Defense Evidence

Evidence Log

Test Environment:
- Hardened DVWA v1.10
- Target Asset: vulnerable-app
- Authentication: admin/password
- Security Level: Low

Total Vulnerabilities Re-tested: 2
Total Successful Exploits Before Patch: 2
Total Successful Exploits After Patch: 0

Verification Results:
✓ SQL Injection blocked
✓ Reflected XSS blocked
✓ Neo4j graph updated

✓ Mitigation relationships verified

Overall Verification Status:
SUCCESS

---

7. Code Patch Summary

Vulnerability	File	Patch Type	Root Cause	Defense Mechanism
SQL Injection	sqli/low.php	Prepared Statements	Unparameterized SQL Query	`mysqli_prepare()` + `mysqli_stmt_bind_param()`
Reflected XSS	xss_r/low.php	Output Encoding	Unencoded User Input	`htmlspecialchars(ENT_QUOTES, UTF-8)`

---

8. Recommendations & Next Steps

Immediate Actions (Completed)

• SQL Injection vulnerability patched using prepared statements

• Reflected XSS vulnerability patched using output encoding

• Source code changes successfully deployed to the hardened DVWA environment

• Post-patch re-testing confirmed both attack vectors are no longer exploitable

• Neo4j knowledge graph updated to reflect mitigation status

Ongoing Security Practices

1. Input Validation

In addition to prepared statements, implement whitelist-based validation for all user-supplied input before processing.

2. Output Encoding

Continue applying context-aware output encoding using functions such as:

htmlspecialchars()

for all user-generated content.

3. Security Headers

Implement additional HTTP security headers:

X-Content-Type-Options: nosniff

X-Frame-Options: DENY

Content-Security-Policy: default-src 'self'

4. Regular Patch Management

Establish a scheduled patch cycle for:

- PHP updates
- Database updates
- Web server updates
- Third-party dependencies

5. Automated Security Testing

Perform periodic security testing using:

- PentAGI
- OWASP ZAP
- Burp Suite
- Static Code Analysis

Long-Term Hardening

1. Upgrade from intentionally vulnerable DVWA configurations to production-grade secure frameworks.
2. Implement a Web Application Firewall (WAF) to provide an additional defensive layer.
3. Enable security logging and monitoring to detect injection attempts and malicious activity.
4. Conduct periodic penetration testing to identify newly introduced vulnerabilities.
5. Provide secure coding training for developers to reduce future coding mistakes.

---

9. Compliance & Attestation

Security Standards Alignment

OWASP Top 10 (2021)

Category	Status
A03:2021 – Injection	Mitigated
A07:2021 – Cross-Site Scripting (XSS)	Mitigated

CWE Mapping

CWE	Description	Status
CWE-89	SQL Injection	Resolved
CWE-79	Cross-Site Scripting (XSS)	Resolved

Defense Verification Attestation

This assessment confirms that:

1. The identified SQL Injection vulnerability has been successfully remediated through prepared statements and parameter binding.
2. The identified Reflected XSS vulnerability has been successfully remediated through output encoding using htmlspecialchars().
3. Post-patch verification testing demonstrated that the original exploit payloads no longer produce successful exploitation outcomes.
4. The Neo4j knowledge graph was updated to reflect the mitigated state of the application and associated remediation controls.
5. The hardened application achieved an exploitation effectiveness rate of **0%**, compared to **100% before patching**.

---

8. Conclusion

The security hardening process successfully eliminated all confirmed critical vulnerabilities identified during the assessment.

Re-testing using the original attack payloads demonstrated that:

- SQL Injection attacks no longer return database records.
- Reflected XSS payloads are rendered as harmless text.
- Exploitation effectiveness decreased from 100% to 0%.

The implemented controls address the root causes of the vulnerabilities at the source-code level and provide a measurable improvement in the overall security posture of the application.

Final Security Status

Control Area	Status
SQL Injection	Mitigated
Reflected XSS	Mitigated
Re-Testing	Passed
Neo4j Verification	Passed
Overall Assessment	SECURE